ServiceNOW Interview Challenge

David Brace | brace.dave@gmail.com | (925) 785 8404
Github repo: https://github.com/Asana/node-asana

Challenge Part #1

Preflight thoughts: In many previous coding exams/challenges, I've spent a lot of time working with buggy and poorly documented API's… even though the vendor claims their API's are beautiful, stable and well documented. Troublesome API's are painful and soak up time; they cause developer frustration and friction. I've learnt to do a lot of early checking of API Tokens, credentials, endpoints and UID's… to ensure they are well understood and are validated as working as advertised. - My methodology is to validate this for the Asana programmatic experience before going any deeper.

1). I first created a Personal Access Token in my asana account.

- I checked that the ASANA_PAT is valid and working via has a basic curl endpoint shell script test:
- code: `curl -X GET https://app.asana.com/api/1.0/users/me -H "Authorization: Bearer 2/1214958591221326/1214948921564467:841bf9cf1e8f596d56ac7036eaca95b2"`

Not off to a good start… as this initially failed because the docs did not clearly explain that the words "Authorization: Bearer" must be present in the token string. I discovered this via a simple Gemini chat, pasting the original curl code error message.

I used the above basic endpoint because it doesn't require the explicit "`workspace_gid`" string of my account (which is an unknown variable at this early point). In this endpoint, the workspace_gid field can be "`me`", or the gid of a user. But "`me`" allows you to discover your own workspace_gid !!

2). I received many errors relating to *_gid failures. After some quick research (using Claude desktop app) I discovered that there at least 3 types of "*ASANA*_gid's".

1. A **workspace** is the top-level container in Asana (e.g. "Acme Corp"). Its GID identifies the workspace itself.
2. A **user** is a person, with their own GID.
3. A **workspace membership** is the *relationship object* connecting one user to one workspace — it records whether they're an admin, a guest, view-only, on vacation, etc. It has its *own* GID, separate from both the user GID and the workspace GID.

I then wrote 3 python scripts to exercise all API and REST url endpoints and API's, to discover and identify that all *_gid's were understood, and working. (not reporting 404 errors).

Outputs from to 3 different scripts.:

#1 (python code: a_asana_1.py

Output:

```
{'created_at': '2026-05-19T20:18:06.143Z',
 'gid': '1214958615680543',
 'is_active': True,
 'is_admin': False,
 'is_guest': False,
 'is_view_only': False,
 'user': {'gid': '1214958591221326', 'name': 'David C. Brace'},
 'user_task_list': {'gid': '1214958605584378',
                    'name': 'My Tasks in My workspace',
                    'owner': {'gid': '1214958591221326',
                              'resource_type': 'user'},
                    'workspace': {'gid': '1214958615680522',
                                  'resource_type': 'workspace'}},
 'vacation_dates': None,
 'workspace': {'gid': '1214958615680522', 'name': 'My workspace'}}
```

#2 (python code: b_asana_1.py

Output:

```
User: David C. Brace (orville.wrightt@gmail.com)
----------------------------------------
Available Workspaces (1):
  • Name: My workspace
    GID:  1214958615680522
```

#3 (python code: c_asana_1.py

Output:

```
Authenticated as:
{'email': 'orville.wrightt@gmail.com',
        'gid': '1214958591221326',
        'name': 'David C. Brace',
        'workspaces': [{'gid': '1214958615680522', 'name': 'My workspace'}]}

Workspace memberships:
```

```
{'gid': '1214958615680543',
        'is_active': True,
        'is_admin': False,
        'is_guest': False,
        'workspace': {'gid': '1214958615680522', 'name': 'My workspace'}}
```

I now have all the tokens, API endpoints, *_gids that I need to start building.
All 3 python scripts were a combination of manual hand coded (by me) and some vibe coding. I hand coded them so that I truly understood what was going on with the API's. This was good as I discovered that the API for the asana.ApiClient code is an OpenAPI-generated wrapper, and in its constructor threads in the pool are non-daemon threads. This basically means that the automatically created pool worker threads will sit idle in a wait() on a condition variable, at the end of the code… waiting for tasks to signal a thread close()... which that will never come… so even though the python code runs successfully it hangs at the end and never returns to to shell. Which was too annoying for me and unclean. So I fixed it by doing a low level `api_client.pool.close()` tidy-up call and the end of the code.

3). I then created a new python script that merged all 3 python scripts into 1 single script, so that I can do all 3 tests in a single run.
The new code is called: `asana_agent.py`
This helps me when things start breaking, because I can very quickly diagnose whether anything has changed in the state of my credentials.

4). From here, I expanded my objective…
I really wanted my Asana solution to be more "naturally conversationally interactive" and leverage the intelligence and power of an agentic framework to enable me to prompt complex chat instructions rather than having to adhere to rigid input string rules (or fuzzy summarized guesses) that basic REPL enforces. I wanted to be able to do complex operations that my code didn't explicitly have coded in, but the agentic harness can figure out how to make it do; via agentic chain-of-throught, reasoning and planning. - Which the basic REPL loop cannot do without a lot of complex API/SDK coding.

I considered a few architectural options, and the top 2 candidates were…

- migrating the code to Agentic SKILLS (for Claude Code)
- creating an MCP server with 3 core Tools. I chose to build an MCP server.

I decided to go with the MCP server option. I wanted to leverage my basic asana_agent.py code as the starting point, and create an MCP server from that. I wanted to use python FastMCP as the MCP server core and have it communicate in stdio mode to Claude Code.

Vibe coding prompt:

```
Evaluate my initial code base asana_agent.py and create an MCP server as completely new code. Don't change or update the existing @asana_agent.py code. Make it a new module.
```

```
The MCP server should have base 3 tools that map into the core capabilities of asana_agent.py
1.    Create an Asana task
2.    Display all Asana asks
3.    Close a Task

The MCP server should load the API token, credentials, workspace id, project id from the .env
The MCP server should use stdio mode
The MCP server should be able to work for Claude Code and OpenAI Codex
The MCP server will run in a linux and Windows10 system
```

4). After testing, I added the following…

- a new (4th) MCP tool to allow modifying Asana task details. - modify_asana_task
- A 4th column to the tabular output that shows the Due Date of each task

Environment note:

- For planning code design, I'm leveraging Claude Code, CodeX and Gemini. I play these 3 off each other when I'm not comfortable with the design approach that one gives me, or when I get errors/failures I ask a different tool to review.
- I am developing my code in OpenAI Codex. My Codex IDE is running on my Windows10 desktop
- I'm pushing all code to my github repo: `https://github.com/Asana/node-asana`
    - See the Readme in the repo for more details
- Im using UV as my python package / environment / project management tool
    - Example uv commands (if you're not familiar with uv)
    - `uv add <python_dependency>`
    - `uv tree`
    - `uv run asana_agent.py`
    - `uv run asana_REPL_copilot.py`
- I'm testing and running my code on a Linux system (Kbuntu 25.10) with python 3.13.7
- I'm testing / running the final agentic workflow code (Asana MCP server) in Claude Code CLI on Linux

To run and work with my final Asana solution….

1. ssh into a linux server
2. Clone the repo: `git clone git@github.com:Asana/node-asana.git`

3. Install the required python modules: `uv sync`
4. Run Claude Code (assumed to be installed on the linux system)
5. Ask claude to: “add the MCP server `asana_mcp_server.py` to the Claude Code config”
6. Check that the asana MCP server was successfully added by doing the following…
    a. /mpc > check that asana is showing and has a status of: √ connected · 4 tools
    b. /mcp > asana > view tools
7. If needed, exit Claude Code and restart it to force a refresh of the MCP configuration. Then redo the above checks in #6
8. Create a `.env` file in the main code directory and enter the following info below…
    a. Note: This will configure the MCP server to chat with my asana project. Dont worry, I will nuke this info…
        - `ASANA_TOKEN=2/1214958591221326/1214948921564467:841bf9cf1e8f596d56ac7036eaca95b2`
        - `ASANA_WORKSPACE_GID=1214958615680522`
        - `ASANA_PROJECT_GID=1214982995972383`
        - `ASANA_API_BASE=https://app.asana.com/api/1.0`
        - `ASANA_IMPLEMENTED_SECTION_NAME=Implemented`
    b. Note: You can use your own personal Asana account and info if you want.
        - If you do, you should use my python pre-flight check tool `svcnow_a_challenge.py` to discover all the key info above. It requires that you to set the shell environment variable ASANA_TOKEN
            - At the shell, type: export ASANA_TOKEN=”paste_your_personal_asana_token_here”
            - then … `uv run svcnow_a_challenge.py`

Start chatting with Claude about your asana project tasks…
Example chats:

> show me all the open asana tasks
> close the 4 tasks due today
> add 5 new tasks with random descriptions, add a unique random number at the end of each description, precede each number with a “#” sign, give each task the due date of 7 days from today
> modify the Task title of task 003 to “Test modifying a task from Claude Code” and change the due date to 10 days from today.
> close the 3 oldest tasks

Challenge Part #1 complete